

PROYECTO CARRETERAS - MANUAL TÉCNICO Y DE ARQUITECTURA

1. Arquitectura del Sistema

El proyecto está desarrollado en Unity y se apoya fuertemente en el sistema Cinemachine para la gestión de cámaras y en componentes de interfaz gráfica (UI Line Renderer) para el dibujado vectorial de topografía. El sistema sigue un patrón de diseño basado en componentes acoplados mediante eventos y referencias directas en el Inspector.

2. Gestión de Cámaras y Movimiento

El control de la vista del usuario se administra mediante un gestor de prioridades.

- **Gestión Centralizada:** El script CambiarCamara.cs controla las transiciones ajustando la propiedad Priority de las Virtual Cameras de Cinemachine (Cámara Libre, Conductor, Dron y Abscisas).
- **Sincronización de Estado:** El script MenuController.cs incluye funciones de teletransporte para garantizar que el avatar del jugador (CapsulaJugador) o el vehículo compartan las mismas coordenadas al intercambiar vistas, evitando la pérdida de continuidad.
- **Control del Vehículo:** El avance sobre la vía se ejecuta manipulando un CinemachineDollyCart mediante el script MoverConTeclasSoloSiActiva.cs. Este componente permite invertir el sentido de la ruta e ingresar a una abscisa específica (saltando a múltiplos de 20 metros) leyendo los BoxCollider de la pista.

3. Lectura y Gráfica de Datos Topográficos

El simulador extrae y renderiza perfiles de terreno dinámicamente.

- **Ingesta de Datos:** TerrenoLoader.cs procesa un archivo CSV, transformando las coordenadas separadas por punto y coma en diccionarios de datos clasificados por abscisas.
- **Renderizado UI:** Los scripts LineaTerrenoUI.cs y LineaEditable.cs transforman los vectores de ingeniería en puntos normalizados dentro de un UILineRenderer.
- **Interacción de Perfiles (Corte y Relleno):** El sistema calcula matemáticamente los chaflanes en tiempo real utilizando la intersección de vectores basándose en los taludes de corte y relleno ingresados por el usuario. La interacción en el modelo 3D se dispara mediante un Raycast en PosteClickManager.cs, que detecta colisiones exclusivas con objetos de la etiqueta "Poste".

4. Despliegue

El aplicativo está configurado para compilarse bajo la arquitectura WebGL. Los ajustes de iluminación (Bias) y plano de corte de las cámaras (Clipping Planes) han sido optimizados para evitar el "Shadow Acne" y parpadeo de texturas durante su ejecución nativa en navegadores web a través de la plataforma itch.io.

Link directo del aplicativo:

<https://piper-ka.itch.io/proyecto-carreteras>